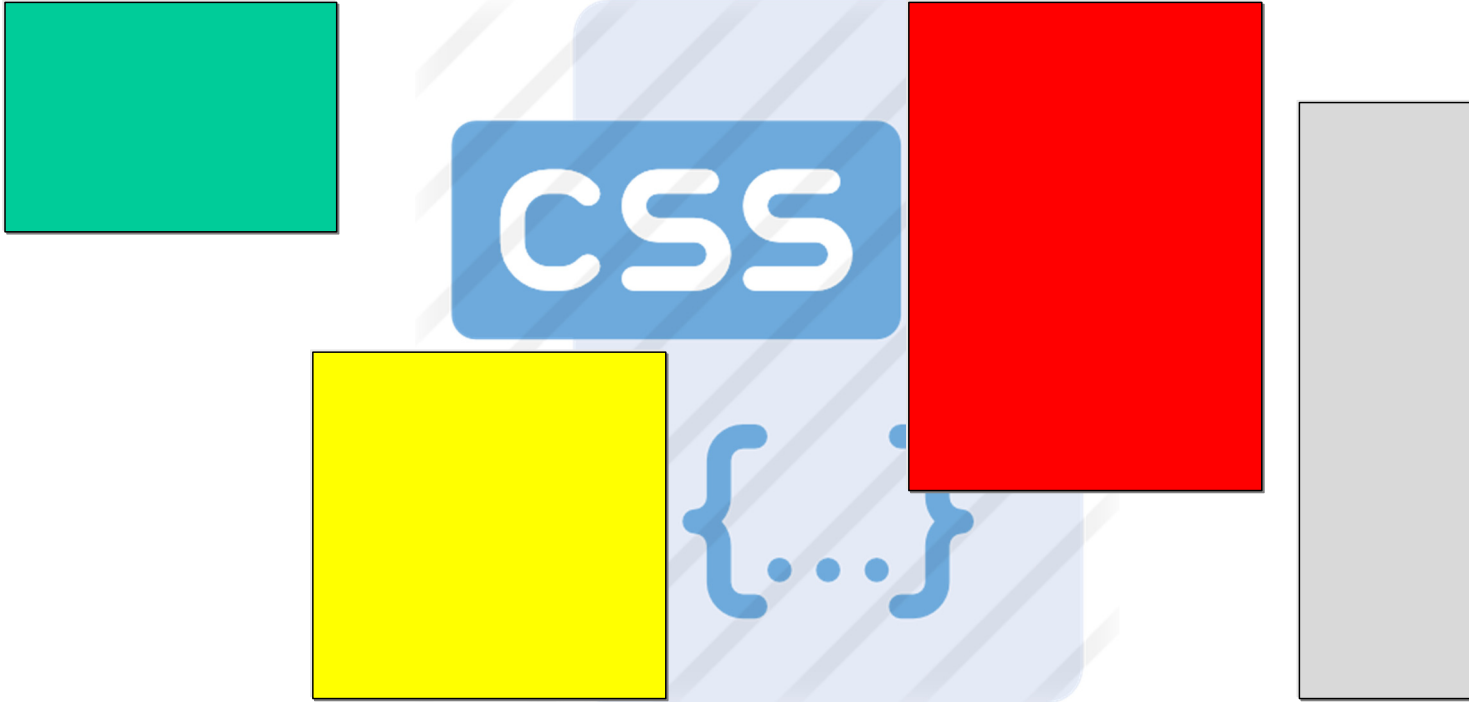
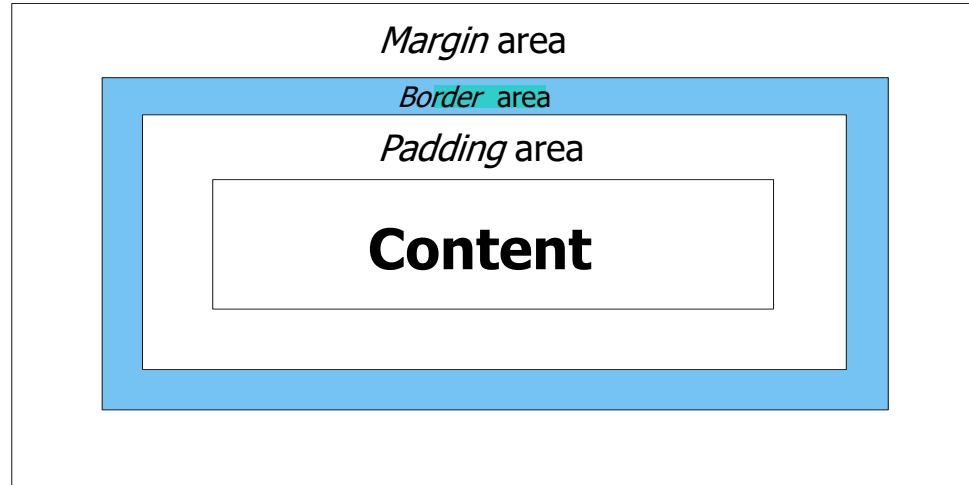


Page Layout with CSS



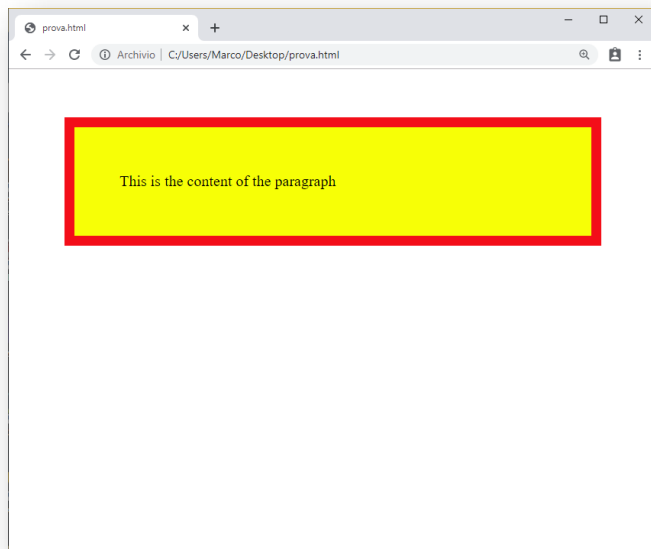
The *box model*

Any HTML element occupies a **rectangular area**, called *box*



Any box is subdivided into a **content** area and into possible **padding**, **border**, and **margin** areas

An example



```
...  
...  
<style type="text/css">  
  p {  
    background: yellow;  
    padding: 3em;  
    border: solid red 10px;  
    margin: 50px;  
  }  
</style>  
</head>  
<body>  
  <p>  
    This is the content  
    of the paragraph  
  </p>  
</body>  
</html>
```

Padding properties

padding-top, padding-bottom, padding-left **and** padding-right

Specify the width of the padding areas

padding alone sets the values for the four sides at once (e.g., padding: 3em)

- If only one value is specified, it is applied to all four sides
- If two values are specified, they refer, respectively, to the top and bottom sides, and to the left and right sides
- If three values are specified, they refer, respectively, to the top side, to the left and right sides, and to the bottom side
- If four values are specified, they refer, respectively, to the top, right, bottom, and left sides

Border properties...

`border-top-width`, `border-right-width`, `border-bottom-width`,
and `border-left-width`

Specify the width of border areas

"qualitative" values `thin`, `medium`, and `thick` or explicit values; `border-width` sets the value for all four sides

`border-top-color`, `border-right-color`, `border-bottom-color`,
and `border-left-color`

Specify the color of the border areas

`border-color` sets the value for all four sides

... border properties

`border-top-style`, `border-right-style`, `border-bottom-style`,
and `border-left-style`

Specify the "style" of border areas

Values **none** (default), `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset`, `outset`; `border-style` sets the value for all four sides

`border-top`, `border-right`, `border-bottom`, **and** `border-left`

Summarize the properties of border areas (e.g., `border-bottom: thick solid red`)

`border`

sets the same values for the three properties for all four sides
(e.g., `border: solid red 3px`)

Margin properties

margin-top, margin-bottom, margin-left, **and** margin-right

Specify the width of the margin areas

margin alone sets the value for the four sides at once

(e.g., margin: 3em)

Same “rules” as for padding areas (e.g., if two values are specified, they refer, respectively, to the top and bottom sides, and to the left and right sides)

Box types

Block-level boxes

In “visual media” (web browsers, printers, ...) are displayed as separate “blocks” (e.g., `<p>` element)

Inline-level boxes

In visual media, they are not displayed as separate blocks (e.g., `<em>` element)

display property

Allows to specify how an HTML element must be displayed by a visual user agent

E.g. `p { display: inline }`
`em { display: block }`
`img { display: none }`

Box size

width and height properties

Specify the width and height of the content area of block boxes

Both pixel and percentage values

Example:

```
p {  
    width: 100px;  
    height: 200px;  
}
```

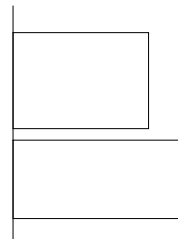
min-width, max-width, min-height, and max-height properties

Specify the minimum and maximum size of a box

Normal flow positioning

Block boxes

- Are vertically arranged, one after the other, from top to bottom
- The vertical distance between two adjacent boxes is determined by the `margin-top` property
- The left edge (or right edge, for right alignment) of the box is contiguous to the left (or right) edge of the container block (but the body usually has a default margin)



Inline boxes

Are horizontally arranged one after the other, starting from the top; margin, border, and padding areas are taken into account

Box positioning

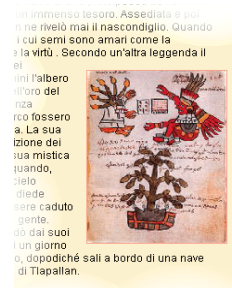
Float positioning

The box is first arranged according to normal flow positioning, and then is **shifted** to the left or to the right

float property

left, right, and none values

E.g., `img { float: right }`



Absolute positioning

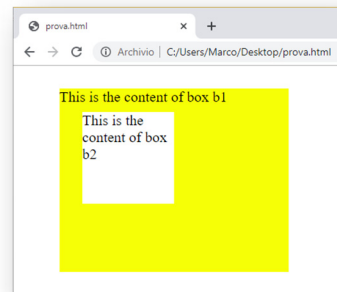
The box is positioned with respect to its container

position property with value absolute

Example:

```
#b1 {  
  background: yellow;  
  width: 250px; height: 200px;  
  position: absolute;  
  top: 25px; left: 50px;  
}  
#b2 {  
  background: white;  
  width: 100px; height: 100px;  
  position: absolute;  
  top: 25px; left: 25px;  
}
```

```
<div id="b1">  
  This is the content of box b1  
  <div id="b2">This is the content  
    of box b2  
  </div>  
</div>
```

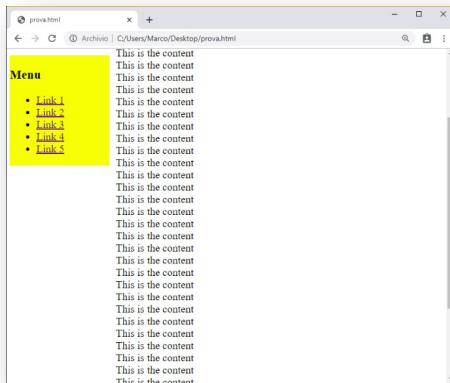


Fixed positioning

The box is positioned with respect to the **browser window** - position property with value `fixed`

The box does not move during page scrolling

Example:



```
body { margin: 5px; }  
  
#menu {  
  position: fixed;  
  width: 150px;  
  background: yellow;  
  top: 10px;  
}  
  
#content {  
  background: white;  
  margin-left: 160px;  
  position: absolute;  
  top: 10px;  
}
```

Relative positioning

The box is arranged according to the **normal flow** positioning, but it is possible to specify **offsets** (**shifts** to the left, to the right, up, and down) - `position` property with value `relative`

Unlike boxes positioned in an absolute way (which, if a width is not specified, occupy only the strictly necessary space), boxes arranged according to the relative positioning occupy all the available space in width

Example:

Here is the equation: $x^2 - 4 = 0$

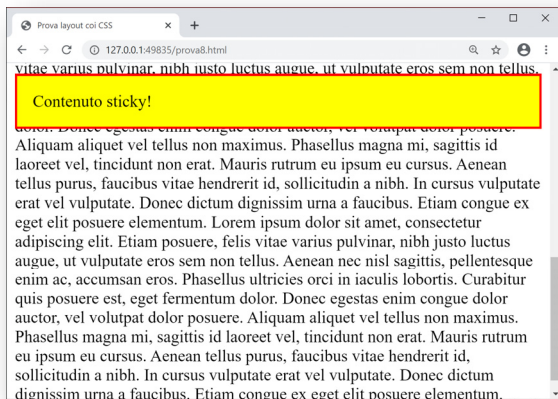
```
.superscript {  
  position: relative;  
  font-size: 0.5em;  
  top: -0.7em;  
}  
<p>  
  Here is the equation:  
  x<span class="superscript">2</span> - 4 = 0  
</p>
```

Sticky positioning

The box is arranged according to **page scrolling**

The "sticky" element moves with the page as long as there is space, then it remains fixed at the top and/or bottom and/or left and/or right (inside its container)

Example:



```
.sticky {  
    position: sticky;  
    top: 10px;  
    padding: 15px;  
    background-color: #ffff00;  
    border: 2px solid #ff0000;  
}
```

Grid positioning

The box is arranged within a **grid-based layout** system, with rows and columns

A grid is composed of a parent element and one or more child elements

Example:

1		2
3	4	5
6	7	8

```
.grid-container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  gap: 10px;  
  background-color: #2196F3;  
  padding: 10px;  
}  
  
.item1 {  
  grid-column-start: 1;  
  grid-column-end: 3;  
}
```


Overlaps

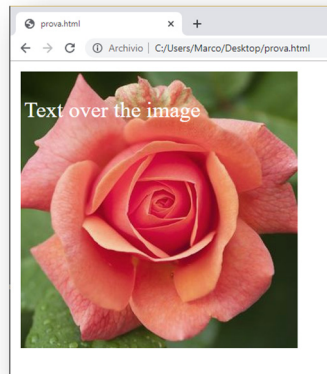
The `z-index` property allows to specify the **overlap order** of boxes that occupy the same area

The box with the highest `z-index` will be displayed in the foreground, the one with the lowest `z-index` will be displayed in the background - Example:

```
#description {  
  position: absolute;  
  top: 20px; left: 20px;  
  color: white;  
  font-size: 2em;  
  z-index: 2;  
}  
  
#im {  
  position: absolute;  
  top: 15px; left: 15px;  
  z-index: 1;  
}
```

```
<p id="description">  
  Text over the image  
</p>  

```



Overflow

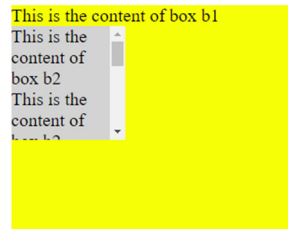
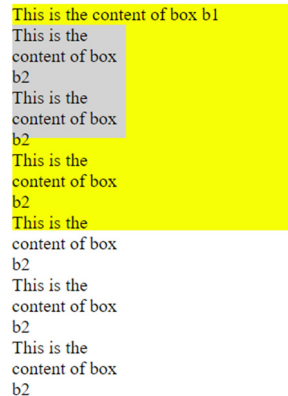
If a (block) box is too small for its content, the content will be displayed outside

The `overflow` property defines how the content of a box must be managed

Values `visible` (default), `hidden`, `scroll`, and `auto`

Example:

```
#b2 {  
  background: lightgrey;  
  width: 100px; height: 100px;  
  position: absolute;  
  top: 25px; left: 25px;  
  overflow: auto;  
}
```



Clipping regions

A clipping region defines an area within which the content of a box is visible
– different shapes

Example:

```
#circ {  
  position: absolute;  
  width: 100px;  
  background: lightblue;  
  padding: 10px;  
  clip-path: circle(20%);  
}
```

